

LAPORAN TUGAS 3 (1 - 3)
SISTEM OPERASI



Dosen Pengampu:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Fagil Lola Prihernando (2410651031)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi prosesor multicore telah membawa paradigma baru dalam pemrograman komputer, dimana eksekusi paralel menjadi solusi optimal untuk meningkatkan kinerja aplikasi. Sistem operasi modern memanfaatkan konsep multithreading untuk menjalankan beberapa tugas secara bersamaan, memaksimalkan utilisasi resource hardware yang tersedia. Praktikum ini mengeksplorasi implementasi konkurensi melalui tiga skenario berbeda: kalkulator paralel, pemrosesan file, dan pola producer-consumer. Ketiga model ini merepresentasikan tantangan nyata dalam pemrograman paralel seperti sinkronisasi, race condition, dan manajemen memory shared. Pemahaman mendalam tentang mekanisme thread, mutex, dan komunikasi antar proses menjadi fondasi essential dalam pengembangan software modern yang scalable dan efficient. Melalui eksperimen praktis ini, mahasiswa tidak hanya memahami teori konsep konkurensi tetapi juga mengalami langsung kompleksitas dan solusi dalam pemrograman paralel yang menjadi standar industri saat ini.

1.2 RUMUSAN MASALAH

- 1 Bagaimana mengimplementasikan pemrograman paralel menggunakan thread untuk mengeksekusi operasi matematika secara simultan?
- 2 Bagaimana membagi tugas pemrosesan file teks menjadi beberapa segment dan melakukan penghitungan kata secara paralel?
- 3 Bagaimana mengelola sinkronisasi akses data shared antara producer dan consumer menggunakan mekanisme mutex?

1.3 TUJUAN

- 1 Memahami konsep dan implementasi multithreading dalam bahasa pemrograman C
- 2 Menganalisis perbandingan performa antara eksekusi singlethread dan multithread
- 3 Mengimplementasikan mekanisme sinkronisasi untuk mencegah race condition pada akses data shared

BAB II

PEMBAHASAN

2.1 PRAKTIK TUGAS 1

Program kalkulator paralel ini merepresentasikan implementasi konkurensi sophisticated yang memanfaatkan arsitektur multithreading untuk mengeksekusi operasi aritmetika secara simultan. Dengan mendefinisikan struktur data terpusat `KalkulatorData` yang mengemas parameter input, hasil komputasi, dan metadata operasi, program menciptakan empat thread independen yang masing-masing menjalankan fungsi spesialis—penjumlahan, pengurangan, perkalian, dan pembagian. Mekanisme ini memungkinkan paralelisme sejati dimana semua komputasi dapat berjalan bersamaan pada prosesor multicore, secara signifikan meningkatkan throughput komputasi dibanding pendekatan sekuensial. Setiap thread menerima pointer ke struktur data terisolasi melalui parameter generik `void\*` yang kemudian dilakukan type casting, memastikan isolasi memori tanpa memerlukan sinkronisasi kompleks. Main thread berfungsi sebagai coordinator yang menginisialisasi data, meluncurkan worker threads menggunakan `pthread\_create`, dan melakukan blocking dengan `pthread\_join` untuk menjamin kelengkapan eksekusi sebelum menampilkan konsolidasi hasil akhir. Desain ini tidak hanya mendemonstrasikan efisiensi pemrosesan paralel untuk tugas-tugas independen, tetapi juga mengimplementasikan penanganan error elegan untuk kasus pembagian oleh nol, menciptakan sistem komputasi robust yang mengoptimalkan utilisasi resources hardware modern.

2.1.1 Membuat Kalkulator Paralel

```
asus@LAPTOP-VRS206DG:~$ nano kalkulator_paralel.c
asus@LAPTOP-VRS206DG:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -pthread
asus@LAPTOP-VRS206DG:~$ ./kalkulator_paralel
=== KALKULATOR PARALEL ===
Masukkan angka pertama: 10
Masukkan angka kedua: 15

Memulai perhitungan paralel...

Thread Penjumlahan: 10.00 + 15.00 = 25.00
Thread Pengurangan: 10.00 - 15.00 = -5.00
Thread Perkalian: 10.00 * 15.00 = 150.00
Thread Pembagian: 10.00 / 15.00 = 0.67

=== HASIL AKHIR ===
Penjumlahan: 25.00
Pengurangan: -5.00
Perkalian: 150.00
Pembagian: 0.67
asus@LAPTOP-VRS206DG:~$ |
```

2.1.2 Penejelasan Proses Kompilasi dan Eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File kalkulator_paralel.c diproses oleh preprocessor untuk menambahkan kode dari header files seperti stdio.h, pthread.h, dan stdlib.h
- Compilation: Kode C dikompilasi menjadi bahasa assembly menggunakan GCC compiler dengan opsi -pthread
- Assembly: Kode assembly diubah menjadi object code dalam format biner
- Linking: Object code di-link dengan library pthread dan standard C library untuk membuat executable
- Output: Menghasilkan file executable kalkulator_paralel yang siap dijalankan di terminal

B. PROSES EKSEKUSI:

- Program Start: User menjalankan executable ./kalkulator_paralel di terminal
- Input Phase: Program menampilkan menu dan meminta input dua angka dari user melalui scanf()
- Memory Setup: Sistem mengalokasikan memory untuk struktur data KalkulatorData dan thread identifiers
- Data Initialization: Nilai input user disalin ke empat struktur data terpisah untuk setiap operasi matematika

C. THREAD CREATION & EXECUTION:

- Thread Spawning: Program membuat empat thread paralel menggunakan pthread_create()
- Parallel Computation: Setiap thread menjalankan fungsi berbeda secara bersamaan:
 - Thread 1: Melakukan operasi penjumlahan
 - Thread 2: Melakukan operasi pengurangan
 - Thread 3: Melakukan operasi perkalian
 - Thread 4: Melakukan operasi pembagian dengan pengecekan pembagi nol
- Real-time Output: Setiap thread langsung menampilkan hasil perhitungannya saat selesai

D. SYNCHRONIZATION PHASE:

- Thread Joining: Main program menunggu semua thread menyelesaikan tugasnya menggunakan `pthread_join()`
- Result Collection: Program mengumpulkan hasil akhir dari semua struktur data
- Data Consolidation: Hasil dari empat operasi matematika disiapkan untuk ditampilkan

E. FINAL OUTPUT & CLEANUP:

- Summary Display: Menampilkan ringkasan akhir semua operasi dalam format terstruktur
- Resource Release: Sistem secara otomatis membersihkan memory dan resources thread
- Program Termination: Kembali ke terminal dengan return value 0, menandakan eksekusi sukses

F. KODE C:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

// Struktur untuk menyimpan data yang dibagikan ke thread
typedef struct {
    double angka1;
    double angka2;
    double hasil;
    char* operasi;
} KalkulatorData;

// Fungsi untuk penjumlahan
void* tambah(void* arg) {
    KalkulatorData* data = (KalkulatorData*) arg;
    data->hasil = data->angka1 + data->angka2;
    data->operasi = "Penjumlahan";
    printf("Thread Penjumlahan: %.2lf + %.2lf = %.2lf\n", data->angka1, data->angka2,
        data->hasil);
    return NULL;
}
```

```
// Fungsi untuk pengurangan
```

```
void* kurang(void* arg) {  
    KalkulatorData* data = (KalkulatorData*) arg;  
    data->hasil = data->angka1 - data->angka2;  
    data->operasi = "Pengurangan";  
    printf("Thread Pengurangan: %.2lf - %.2lf = %.2lf\n", data->angka1, data->angka2,  
        data->hasil);  
    return NULL;  
}
```

```
// Fungsi untuk perkalian
```

```
void* kali(void* arg) {  
    KalkulatorData* data = (KalkulatorData*) arg;  
    data->hasil = data->angka1 * data->angka2;  
    data->operasi = "Perkalian";  
    printf("Thread Perkalian: %.2lf * %.2lf = %.2lf\n", data->angka1, data->angka2,  
        data->hasil);  
    return NULL;  
}
```

```
// Fungsi untuk pembagian
```

```
void* bagi(void* arg) {  
    KalkulatorData* data = (KalkulatorData*) arg;  
    if (data->angka2 != 0) {  
        data->hasil = data->angka1 / data->angka2;  
        data->operasi = "Pembagian";  
        printf("Thread Pembagian: %.2lf / %.2lf = %.2lf\n", data->angka1, data->angka2,  
            data->hasil);  
    } else {  
        data->hasil = 0;  
        data->operasi = "Pembagian";  
        printf("Thread Pembagian: Error! Pembagian dengan nol.\n");  
    }  
    return NULL;  
}
```

```

int main() {
    double angka1, angka2;

    // Meminta input dari user
    printf("=== KALKULATOR PARALEL ===\n");
    printf("Masukkan angka pertama: ");
    scanf("%lf", &angka1);
    printf("Masukkan angka kedua: ");
    scanf("%lf", &angka2);
    printf("\n");

    // Deklarasi thread dan data
    pthread_t thread_tambah, thread_kurang, thread_kali, thread_bagi;
    KalkulatorData data_tambah, data_kurang, data_kali, data_bagi;

    // Inisialisasi data untuk setiap thread
    data_tambah.angka1 = data_kurang.angka1 = data_kali.angka1 = data_bagi.angka1 =
angka1;
    data_tambah.angka2 = data_kurang.angka2 = data_kali.angka2 = data_bagi.angka2 =
angka2;

    printf("Memulai perhitungan paralel...\n\n");

    // Membuat thread untuk setiap operasi
    pthread_create(&thread_tambah, NULL, tambah, &data_tambah);
    pthread_create(&thread_kurang, NULL, kurang, &data_kurang);
    pthread_create(&thread_kali, NULL, kali, &data_kali);
    pthread_create(&thread_bagi, NULL, bagi, &data_bagi);

    // Menunggu semua thread selesai
    pthread_join(thread_tambah, NULL);
    pthread_join(thread_kurang, NULL);
    pthread_join(thread_kali, NULL);
    pthread_join(thread_bagi, NULL);

    // Menampilkan hasil akhir
    printf("\n=== HASIL AKHIR ===\n");
}

```

```
printf("%s: %.2lf\n", data_tambah.operasi, data_tambah.hasil);
printf("%s: %.2lf\n", data_kurang.operasi, data_kurang.hasil);
printf("%s: %.2lf\n", data_kali.operasi, data_kali.hasil);
printf("%s: %.2lf\n", data_bagi.operasi, data_bagi.hasil);

return 0;
}
```

2.2 PRAKTIK TUGAS 2

Implementasi program multithreading ini mendemonstrasikan paradigma pemrosesan paralel untuk menghitung kata dalam file teks dengan memanfaatkan arsitektur dua thread yang bekerja secara simultan. Program menggunakan struktur data ThreadData yang mengemas segmentasi teks beserta metadata yang diperlukan, dimana setiap thread menjalankan fungsi `hitung_kata_thread` untuk memproses bagian tertentu secara independen. Algoritma penghitungan kata mengimplementasikan finite state machine dengan variabel `in_word` yang mendeteksi transisi antara karakter spasi dan non-spasi, memastikan akurasi perhitungan bahkan untuk kata yang terpotong pada boundary segmentasi. Mekanisme pembagian tugas yang cerdas memastikan pemisahan teks pada boundary kata yang tepat melalui iterasi hingga menemukan whitespace, menghindari race condition tanpa memerlukan sinkronisasi kompleks karena isolasi data yang inherent. Eksekusi paralel kedua thread kemudian disinkronisasi menggunakan `pthread_join` sebelum hasil akhir dikonsolidasi, sementara versi singlethread berfungsi sebagai baseline performa untuk perbandingan komprehensif efisiensi eksekusi yang diukur dengan presisi mikrodetik menggunakan fungsi `clock()`. Program ini tidak hanya mengilustrasikan superioritas throughput komputasi pada sistem multi-core tetapi juga menerapkan best practices dalam manajemen memori dan error handling untuk operasi file I/O.

2.2.1 Membuat File Processing

- Otomatis

```
asus@LAPTOP-VRS206DG:~$ nano file_processing.c
asus@LAPTOP-VRS206DG:~$ gcc file_processing.c -o file_processing -pthread
asus@LAPTOP-VRS206DG:~$ ./file_processing
File contoh.txt berhasil dibuat
=== FILE PROCESSING PARALEL ===
Panjang teks: 311 karakter

=== VERSI MULTITHREAD ===
Thread: Menghitung kata dari indeks 0-157 = 24 kata
Thread: Menghitung kata dari indeks 157-311 = 19 kata
Total kata (multithread): 43
Waktu eksekusi: 0.000604 detik

=== VERSI SINGLETHREAD ===
Total kata (singlethread): 43
Waktu eksekusi: 0.000004 detik

=== PERBANDINGAN ===
Multithread: 43 kata, 0.000604 detik
Singlethread: 43 kata, 0.000004 detik
KESIMPULAN: Singlethread lebih cepat 0.000600 detik
asus@LAPTOP-VRS206DG:~$ |
```

2.2.2 Penjelasan Proses Kompilasi dan Eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File file_processing.c diproses oleh preprocessor untuk menambahkan kode dari header files seperti stdio.h, pthread.h, dan string.h
- Compilation: Kode C dikompilasi menjadi bahasa assembly menggunakan GCC compiler dengan opsi -pthread
- Assembly: Kode assembly diubah menjadi object code dalam format biner
- Linking: Object code di-link dengan library pthread dan standard C library untuk membuat executable
- Output: Menghasilkan file executable file_processing yang siap dijalankan di terminal

B. INISIALISASI PROGRAM:

- Program Start: User menjalankan executable ./file_processing di terminal
- File Creation: Program membuat file contoh.txt otomatis dengan konten teks untuk testing
- File Reading: Membaca seluruh isi file contoh.txt ke dalam memory menggunakan fungsi baca_file()
- Memory Allocation: Mengalokasikan memory untuk menyimpan teks dari file yang dibaca
- Setup Awal: Menghitung panjang teks dan menampilkan informasi dasar program

C. VERSI MULTITHREAD:

- Thread Preparation: Membuat dua thread dan struktur data untuk masing-masing thread
- Text Splitting: Membagi teks menjadi dua bagian di tengah-tengah kata (bukan asal potong)
- Thread Launch: Menjalankan dua thread secara paralel menggunakan `pthread_create()`
- Parallel Counting: Setiap thread menghitung kata di bagiannya sendiri-sendiri:
 - Thread 1: Menghitung kata dari awal sampai tengah teks
 - Thread 2: Menghitung kata dari tengah sampai akhir teks
- Result Combination: Menjumlahkan hasil dari kedua thread setelah selesai

D. VERSI SINGLETHREAD:

- Sequential Execution: Menghitung semua kata dalam teks menggunakan satu thread saja
- Same Algorithm: Menggunakan algoritma penghitungan kata yang sama dengan versi multithread
- Performance Baseline: Hasilnya digunakan sebagai pembanding kecepatan

E. PERBANDINGAN PERFORMANSI:

- Time Measurement: Mengukur waktu eksekusi untuk kedua versi dengan fungsi `clock()`
- Result Display: Menampilkan jumlah kata dan waktu eksekusi kedua versi
- Speed Analysis: Membandingkan mana yang lebih cepat antara multithread dan singlethread
- Conclusion: Menarik kesimpulan berdasarkan perbandingan waktu eksekusi

F. FINALISASI:

- Memory Cleanup: Membersihkan memory yang dialokasikan untuk teks
- Resource Release: Sistem otomatis melepas resources thread dan file handles
- Program Exit: Kembali ke terminal dengan status sukses

G. KODE C:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/stat.h>

// Struktur untuk data thread
typedef struct {
    char* text;
    int start;
    int end;
    int word_count;
} ThreadData;

// Fungsi menghitung kata dalam bagian teks
int hitung_kata_bagian(char* text, int start, int end) {
    int count = 0;
    int in_word = 0;

    for (int i = start; i < end; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t' || text[i] == '\0') {
            if (in_word) {
                count++;
                in_word = 0;
            }
        } else {
            in_word = 1;
        }
    }
}

// Hitung kata terakhir jika teks berakhir tanpa spasi
if (in_word) {
    count++;
}
```

```

    return count;
}

// Fungsi thread untuk menghitung kata
void* hitung_kata_thread(void* arg) {
    ThreadData* data = (ThreadData*) arg;
    data->word_count = hitung_kata_bagian(data->text, data->start, data->end);

    printf("Thread: Menghitung kata dari indeks %d-%d = %d kata\n",
        data->start, data->end, data->word_count);
    return NULL;
}

// Fungsi singlethread (versi biasa)
int hitung_kata_singlethread(char* text) {
    return hitung_kata_bagian(text, 0, strlen(text));
}

// Fungsi baca file
char* baca_file(const char* filename) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        printf("Error: Tidak bisa membuka file %s\n", filename);
        return NULL;
    }

    // Cari ukuran file
    fseek(file, 0, SEEK_END);
    long size = ftell(file);
    fseek(file, 0, SEEK_SET);

    // Allokasi memori untuk teks
    char* text = malloc(size + 1);
    fread(text, 1, size, file);
    text[size] = '\0';

    fclose(file);
}

```

```

    return text;
}

// Fungsi buat file contoh
void buat_file_contoh() {
    FILE* file = fopen("contoh.txt", "w");
    if (file) {
        fprintf(file, "Ini adalah file teks contoh untuk praktikum sistem operasi\n");
        fprintf(file, "Kita akan menghitung jumlah kata dalam file ini\n");
        fprintf(file, "Menggunakan dua thread yang bekerja paralel\n");
        fprintf(file, "Thread pertama menghitung separuh bagian awal\n");
        fprintf(file, "Thread kedua menghitung separuh bagian akhir\n");
        fprintf(file, "Hasilnya akan digabungkan dan dibandingkan dengan versi
singlethread\n");
        fclose(file);
        printf("File contoh.txt berhasil dibuat\n");
    }
}

int main() {
    // Buat file contoh otomatis
    buat_file_contoh();

    // Baca file
    char* text = baca_file("contoh.txt");
    if (!text) {
        return 1;
    }

    int text_length = strlen(text);
    printf("=== FILE PROCESSING PARALEL ===\n");
    printf("Panjang teks: %d karakter\n\n", text_length);

    // Versi MULTITHREAD
    printf("=== VERSI MULTITHREAD ===\n");
    clock_t start_multi = clock();

```

```

pthread_t thread1, thread2;
ThreadData data1, data2;

// Bagi teks menjadi 2 bagian
int mid = text_length / 2;

// Pastikan pembagian tidak memotong kata di tengah
while (mid < text_length && text[mid] != ' ' && text[mid] != '\n') {
    mid++;
}

data1.text = text;
data1.start = 0;
data1.end = mid;

data2.text = text;
data2.start = mid;
data2.end = text_length;

// Buat thread
pthread_create(&thread1, NULL, hitung_kata_thread, &data1);
pthread_create(&thread2, NULL, hitung_kata_thread, &data2);

// Tunggu thread selesai
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

// Gabungkan hasil
int total_multi = data1.word_count + data2.word_count;
clock_t end_multi = clock();
double waktu_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;

printf("Total kata (multithread): %d\n", total_multi);
printf("Waktu eksekusi: %.6f detik\n\n", waktu_multi);

// Versi SINGLETHREAD
printf("=== VERSI SINGLETHREAD ===\n");

```

```

clock_t start_single = clock();

int total_single = hitung_kata_singlethread(text);
clock_t end_single = clock();
double waktu_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;

printf("Total kata (singlethread): %d\n", total_single);
printf("Waktu eksekusi: %.6f detik\n\n", waktu_single);

// Perbandingan
printf("=== PERBANDINGAN ===\n");
printf("Multithread: %d kata, %.6f detik\n", total_multi, waktu_multi);
printf("Singlethread: %d kata, %.6f detik\n", total_single, waktu_single);

if (waktu_multi < waktu_single) {
    printf("KESIMPULAN: Multithread lebih cepat %.6f detik\n", waktu_single -
waktu_multi);
} else {
    printf("KESIMPULAN: Singlethread lebih cepat %.6f detik\n", waktu_multi -
waktu_single);
}

// Bersihkan memori
free(text);
return 0;
}

```

2.3 PRAKTIK TUGAS 3

Implementasi pola producer-consumer ini merepresentasikan solusi konkurensi klasik yang mengatasi masalah sinkronisasi dalam lingkungan multithreading. Program menggunakan buffer circular bersama berukuran tetap yang dilindungi oleh mekanisme mutex untuk mencegah race condition saat akses data simultan. Producer thread secara kontinu menghasilkan item acak dan menyimpannya ke buffer selama kapasitas tersedia, sementara consumer thread mengonsumsi item tersebut dengan melakukan operasi kuadrat pada nilai yang diambil. Kedua thread beroperasi pada kecepatan berbeda—producer lebih cepat dengan delay 100ms dibanding consumer 150ms—menciptakan dinamika alami dalam pengisian dan pengosongan buffer. Implementasi ini menjamin mutual exclusion melalui `pthread_mutex_lock/unlock` yang mengatur akses eksklusif ke struktur data bersama, sementara algoritma circular buffer dengan pointer in dan out memastikan utilisasi memori yang optimal. Program juga menampilkan status real-time melalui fungsi monitoring yang memperlihatkan kondisi buffer, count item, serta posisi pointer, memberikan visualisasi komprehensif tentang interaksi kedua thread selama eksekusi paralel berlangsung hingga mencapai batas maksimal item yang diproses.

2.3.1 Membuat Procedur Consumer

```
asus@LAPTOP-VRS206DG:~$ nano producer_consumer.c
asus@LAPTOP-VRS206DG:~$ gcc producer_consumer.c -o producer_consumer -pthread
asus@LAPTOP-VRS206DG:~$ ./producer_consumer
=== SIMULASI PRODUCER-CONSUMER ===
Buffer size: 10 item
Total item yang akan diproduksi: 20

=== STATUS BUFFER ===
Count: 0/10
Isi buffer: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
In: 0, Out: 0

PRODUCER: Hasilkan item 21 | Buffer: 1/10 item
CONSUMER: Ambil item 21 -> Proses: 21² = 441 | Buffer: 0/10 item
PRODUCER: Hasilkan item 2 | Buffer: 1/10 item
CONSUMER: Ambil item 2 -> Proses: 2² = 4 | Buffer: 0/10 item
PRODUCER: Hasilkan item 14 | Buffer: 1/10 item
CONSUMER: Ambil item 14 -> Proses: 14² = 196 | Buffer: 0/10 item
PRODUCER: Hasilkan item 41 | Buffer: 1/10 item
PRODUCER: Hasilkan item 27 | Buffer: 2/10 item
CONSUMER: Ambil item 41 -> Proses: 41² = 1681 | Buffer: 1/10 item
PRODUCER: Hasilkan item 98 | Buffer: 2/10 item
CONSUMER: Ambil item 27 -> Proses: 27² = 729 | Buffer: 1/10 item
PRODUCER: Hasilkan item 10 | Buffer: 2/10 item
PRODUCER: Hasilkan item 20 | Buffer: 3/10 item
CONSUMER: Ambil item 98 -> Proses: 98² = 9604 | Buffer: 2/10 item
PRODUCER: Hasilkan item 36 | Buffer: 3/10 item
CONSUMER: Ambil item 10 -> Proses: 10² = 100 | Buffer: 2/10 item
PRODUCER: Hasilkan item 9 | Buffer: 3/10 item
PRODUCER: Hasilkan item 86 | Buffer: 4/10 item
CONSUMER: Ambil item 20 -> Proses: 20² = 400 | Buffer: 3/10 item
PRODUCER: Hasilkan item 81 | Buffer: 4/10 item
CONSUMER: Ambil item 36 -> Proses: 36² = 1296 | Buffer: 3/10 item
PRODUCER: Hasilkan item 60 | Buffer: 4/10 item
PRODUCER: Hasilkan item 30 | Buffer: 5/10 item
CONSUMER: Ambil item 9 -> Proses: 9² = 81 | Buffer: 4/10 item
PRODUCER: Hasilkan item 24 | Buffer: 5/10 item
CONSUMER: Ambil item 86 -> Proses: 86² = 7396 | Buffer: 4/10 item
PRODUCER: Hasilkan item 11 | Buffer: 5/10 item
```



```

CONSUMER: Ambil item 20 -> Proses: 20² = 400 | Buffer: 3/10 item
PRODUCER: Hasilkan item 81 | Buffer: 4/10 item
CONSUMER: Ambil item 36 -> Proses: 36² = 1296 | Buffer: 3/10 item
PRODUCER: Hasilkan item 60 | Buffer: 4/10 item
PRODUCER: Hasilkan item 30 | Buffer: 5/10 item
CONSUMER: Ambil item 9 -> Proses: 9² = 81 | Buffer: 4/10 item
PRODUCER: Hasilkan item 24 | Buffer: 5/10 item
CONSUMER: Ambil item 86 -> Proses: 86² = 7396 | Buffer: 4/10 item
PRODUCER: Hasilkan item 11 | Buffer: 5/10 item
PRODUCER: Hasilkan item 97 | Buffer: 6/10 item
CONSUMER: Ambil item 81 -> Proses: 81² = 6561 | Buffer: 5/10 item
PRODUCER: Hasilkan item 32 | Buffer: 6/10 item
CONSUMER: Ambil item 60 -> Proses: 60² = 3600 | Buffer: 5/10 item
PRODUCER: Hasilkan item 21 | Buffer: 6/10 item
PRODUCER: Hasilkan item 79 | Buffer: 7/10 item
CONSUMER: Ambil item 30 -> Proses: 30² = 900 | Buffer: 6/10 item
PRODUCER: Selesai memproduksi 20 item
CONSUMER: Ambil item 24 -> Proses: 24² = 576 | Buffer: 5/10 item
CONSUMER: Ambil item 11 -> Proses: 11² = 121 | Buffer: 4/10 item
CONSUMER: Ambil item 97 -> Proses: 97² = 9409 | Buffer: 3/10 item
CONSUMER: Ambil item 32 -> Proses: 32² = 1024 | Buffer: 2/10 item
CONSUMER: Ambil item 21 -> Proses: 21² = 441 | Buffer: 1/10 item
CONSUMER: Ambil item 79 -> Proses: 79² = 6241 | Buffer: 0/10 item
CONSUMER: Selesai memproses 20 item

=== STATUS BUFFER ===
Count: 0/10
Isi buffer: [86, 81, 60, 30, 24, 11, 97, 32, 21, 79]
In: 0, Out: 0

=== SIMULASI SELESAI ===
asus@LAPTOP-VRS206DG:~$ |

```

2.3.2 Penjelasan Proses Kompilasi dan Eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File producer_consumer.c diproses oleh preprocessor untuk menambahkan kode dari header files seperti stdio.h, pthread.h, dan unistd.h
- Compilation: Kode C dikompilasi menjadi bahasa assembly menggunakan GCC compiler dengan opsi -pthread
- Assembly: Kode assembly diubah menjadi object code dalam format biner
- Linking: Object code di-link dengan library pthread dan standard C library untuk membuat executable
- Output: Menghasilkan file executable producer_consumer yang siap dijalankan di terminal

B. INISIALISASI PROGRAM:

- Program Start: User menjalankan executable ./producer_consumer di terminal
- Random Seed: Program menginisialisasi random number generator dengan waktu sistem
- Buffer Setup: Fungsi init_buffer() dipanggil untuk mengosongkan buffer dan reset semua counter
- Display Info: Program menampilkan informasi konfigurasi buffer dan jumlah item yang akan diproduksi

C. PEMBUATAN THREAD:

- Thread Creation: Program membuat dua thread paralel menggunakan `pthread_create()`
- Parallel Execution: Kedua thread mulai berjalan bersamaan:
 - Thread 1: Producer yang bertugas menghasilkan item
 - Thread 2: Consumer yang bertugas memproses item
- Independent Operation: Setiap thread menjalankan logika yang berbeda secara independen

D. PROSES PRODUCER:

- Item Generation: Producer membuat angka random antara 1-100
- Mutex Lock: Producer mengambil kunci mutex sebelum akses buffer
- Buffer Check: Memeriksa apakah buffer masih ada space kosong
- Item Storage: Jika ada space, item dimasukkan ke buffer dan counter ditambah
- Buffer Full: Jika buffer penuh, producer menunggu dan coba lagi nanti
- Completion: Producer berhenti setelah membuat 20 item

E. PROSES CONSUMER:

- Mutex Acquisition: Consumer mengambil kunci mutex sebelum akses buffer
- Buffer Check: Memeriksa apakah buffer berisi item yang bisa diambil
- Item Processing: Jika ada item, consumer mengambil dan mengkuadratkannya
- Buffer Empty: Jika buffer kosong, consumer menunggu dan coba lagi nanti
- Completion: Consumer berhenti setelah memproses 20 item

F. MEKANISME SINKRONISASI:

- Mutual Exclusion: Mutex memastikan hanya satu thread yang bisa akses buffer pada satu waktu
- Thread Coordination: Kedua thread bekerja dengan kecepatan berbeda (producer lebih cepat)
- Buffer Management: Circular buffer dengan pointer in/out mengatur alur data
- Safe Access: Tidak ada race condition karena protection mutex

G. FINALISASI PROGRAM:

- Thread Joining: Main program menunggu kedua thread selesai dengan `pthread_join()`
- Final Report: Menampilkan status buffer terakhir dan summary eksekusi
- Cleanup: Sistem otomatis melepas resources thread dan memory
- Program Exit: Kembali ke terminal dengan status sukses

H. KODE C:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 10
#define MAX_ITEM 20

// Struktur buffer bersama
typedef struct {
    int buffer[BUFFER_SIZE];
    int count;
    int in;
    int out;
} Buffer;

Buffer shared_buffer;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

// Fungsi producer
void* producer(void* arg) {
    int produced = 0;

    while (produced < MAX_ITEM) {
        // Generate angka random
        int item = rand() % 100 + 1;
```

```

pthread_mutex_lock(&mutex);

// Cek jika buffer penuh
if (shared_buffer.count < BUFFER_SIZE) {
    shared_buffer.buffer[shared_buffer.in] = item;
    shared_buffer.in = (shared_buffer.in + 1) % BUFFER_SIZE;
    shared_buffer.count++;

    printf("PRODUCER: Hasilkan item %d | Buffer: %d/10 item\n", item,
shared_buffer.count);
    produced++;
} else {
    printf("PRODUCER: Buffer penuh! Menunggu...\n");
}

pthread_mutex_unlock(&mutex);
usleep(100000); // Tidur 100ms
}

printf("PRODUCER: Selesai memproduksi %d item\n", MAX_ITEM);
return NULL;
}

// Fungsi consumer
void* consumer(void* arg) {
    int consumed = 0;

    while (consumed < MAX_ITEM) {
        pthread_mutex_lock(&mutex);

        // Cek jika buffer kosong
        if (shared_buffer.count > 0) {
            int item = shared_buffer.buffer[shared_buffer.out];
            shared_buffer.out = (shared_buffer.out + 1) % BUFFER_SIZE;
            shared_buffer.count--;

            // Proses item (kuadratkan)

```

```

    int processed = item * item;

    printf("CONSUMER: Ambil item %d -> Proses: %d² = %d | Buffer: %d/10 item\n",
           item, item, processed, shared_buffer.count);
    consumed++;
} else {
    printf("CONSUMER: Buffer kosong! Menunggu...\n");
}

pthread_mutex_unlock(&mutex);
usleep(150000); // Tidur 150ms (lebih lambat dari producer)
}

printf("CONSUMER: Selesai memproses %d item\n", MAX_ITEM);
return NULL;
}

// Inisialisasi buffer
void init_buffer() {
    shared_buffer.count = 0;
    shared_buffer.in = 0;
    shared_buffer.out = 0;

    for (int i = 0; i < BUFFER_SIZE; i++) {
        shared_buffer.buffer[i] = 0;
    }
}

// Fungsi tampilkan status buffer
void tampilkan_buffer() {
    printf("\n=== STATUS BUFFER ===\n");
    printf("Count: %d/10\n", shared_buffer.count);
    printf("Isi buffer: [");

    for (int i = 0; i < BUFFER_SIZE; i++) {
        printf("%d", shared_buffer.buffer[i]);
        if (i < BUFFER_SIZE - 1) printf(", ");
    }
}

```

```

    }

    printf("]\n");
    printf("In: %d, Out: %d\n\n", shared_buffer.in, shared_buffer.out);
}

int main() {
    pthread_t prod_thread, cons_thread;

    // Seed random number generator
    srand(time(NULL));

    // Inisialisasi buffer
    init_buffer();

    printf("=== SIMULASI PRODUCER-CONSUMER ===\n");
    printf("Buffer size: %d item\n", BUFFER_SIZE);
    printf("Total item yang akan diproduksi: %d\n\n", MAX_ITEM);

    // Tampilkan buffer awal
    tampilkan_buffer();

    // Buat thread producer dan consumer
    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    // Tunggu kedua thread selesai
    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    // Tampilkan buffer akhir
    tampilkan_buffer();

    printf("=== SIMULASI SELESAI ===\n");

    return 0;
}

```

BAB III

PENUTUP

3.1 KESIMPULAN

Praktikum sistem operasi ini telah berhasil mendemonstrasikan implementasi konkurensi melalui tiga skenario pemrograman paralel. Kalkulator paralel membuktikan bahwa tugas-tugas independen dapat dieksekusi secara simultan dengan significant performance improvement. Pemrosesan file teks mengajarkan strategi pembagian workload yang optimal dengan mempertimbangkan boundary kata untuk memastikan akurasi hasil. Pola producer-consumer mengilustrasikan pentingnya mekanisme sinkronisasi dalam mengelola shared resource, dimana mutex menjadi solusi efektif untuk mencegah race condition. Hasil eksperimen menunjukkan bahwa pemrograman paralel memang menawarkan throughput yang lebih tinggi, namun juga memerlukan pertimbangan matang terhadap kompleksitas sinkronisasi dan manajemen memory. Penguasaan konsep konkurensi ini menjadi kompetensi krusial dalam menghadapi era komputasi modern yang didominasi oleh arsitektur multicore dan distributed systems.